VibeLife Ders Kitabi

Cilt 1: Kod okuma sirasi, temel dosyalar ve calisma yontemi

Flutter + Dart + Firebase uygulamasini kendi kodlari uzerinden ogrenme rehberi

Olusturulma tarihi: 14.07.2026

Icindekiler

1. Projenin kimligi: pubspec.yaml
2. Uygulamanin baslangici: main.dart
3. Firebase baslatma ve guvenlik
4. Kucuk dosyalardan Dart ogrenme
5. Ana kabuk ve alt sekmeler: feed_page.dart
6. Gonderi paylasma akisi
7. Sosyal servis katmani
8. Robot sistemi
9. Firestore kurallarini okuma
10. Buyuk dosyalar icin okuma haritasi

Bu kitabi nasil calisacaksin?

> **Calisma mantigi**
> Kodlari roman gibi bastan sona okumaya calisma. Her bolumde once amaci oku, sonra kod parcasi uzerinde isaretleri ve kelimeleri tek tek takip et. Takildigin satiri AI'ye 20-30 satirlik kucuk parcayla sor.

Onerilen sira

1. pubspec.yaml ile projenin paketlerini tanima.
2. main.dart ile uygulamanin nasil basladigini ogrenme.
3. firebase_bootstrap.dart ile Firebase baslatma mantigini anlama.
4. theme_controller.dart, vibe_snackbar.dart ve tarih_saat.dart gibi kucuk dosyalarla Dart dilini pekistirme.
5. feed_page.dart ile ana sekme sistemini takip etme.
6. newpost_page.dart ile bir kullanici aksiyonunun Firestore'a nasil yazildigini gorme.
7. sosyal_servis.dart ve sosyal_core.dart ile servis katmanini okuma.
8. robot_servis.dart ile is kurali ve oyun mekanigini cozme.
9. firestore.rules ile guvenlik mantigini ayri bir katman olarak okuma.

Kod sembolleri sozlugu

Kavram	Bu projede nasil okunur?
`;`	Dart'ta cumle sonudur. Cogu kod satiri noktalı virgulle biter.
`.`	Bir nesnenin icindeki ozellige veya metoda erisir. FirebaseAuth.instance.currentUser gibi.
`()`	Fonksiyon veya constructor cagrisi yapar. runApp(...) gibi.
`{}`	Kod blogu, class govdesi veya named argument grubu olabilir; bulundugu yere gore okunur.
`[]`	Liste veya koleksiyon erisimi icin kullanilir. data['puan'] gibi map erisimi de olabilir.
`<>`	Generic tip belirtir. Future<Widget> gelecekte Widget donecek demektir.
`?`	Null olabilir anlamina gelir. String? degeri bazen bos olabilir.
`!`	Null degil varsayimi yapar. Dikkatli okunmasi gerekir.
`=>`	Tek satirlik fonksiyon govdesidir; soldaki fonksiyon sagdaki ifadeyi dondurur.
`async`	Fonksiyon icinde await kullanilabilecegini ve Future donecegini gosterir.
`await`	Asenkron islemin bitmesini bekler.
`final`	Degisken bir kez atanir, sonra degismez.
`const`	Derleme zamaninda sabit deger veya sabit widget olusturur.
`required`	Named argument'in zorunlu oldugunu soyler.
`override`	Ust siniftan gelen metodu burada yeniden yaziyoruz demektir.

Buyuk dosyalara girmeden once

- profile_page.dart, messages_page.dart ve user_profile_page.dart dosyalarini en sona birak.
- Her okumada sadece 20-30 satir al. Daha fazlasi yeni baslayan icin gereksiz yuk olur.
- Bir fonksiyonu okurken once girislerini, sonra kontrolleri, sonra Firebase yazmalarini, en sonda UI mesajlarini ayir.
- Anlamadigin sembol icin once bu sayfadaki sozluge bak; sonra ayni sembolun baska dosyada nasil kullanildigini ara.

1. Projenin kimligi: pubspec.yaml

> **Bu bolumun amaci**
> Bir Flutter projesinde hangi paketlerin kullanildigini ve uygulamanin hangi varliklari yukledigini once buradan anlarsin.

Bu bolumde sunlari ogren

- Flutter projesinin paket mantigini okumak.
- Firebase, Lottie ve SharedPreferences paketlerinin gorevini ayirt etmek.
- assets bolumunun uygulamadaki animasyon ve avatarlarla iliskisini gormek.

Paketler ve varliklar pubspec.yaml:1-73

Bu dosya kod degil YAML formatidir. Girintiler anlamlidir; bir satirin hangi bolume ait oldugunu bosluklar belirler.

```
  1 | name: vibe_life
  2 | description: "A new Flutter project."
  3 | # The following line prevents the package from being accidentally published to
  4 | # pub.dev using `flutter pub publish`. This is preferred for private packages.
  5 | publish_to: 'none' # Remove this line if you wish to publish to pub.dev
  6 |
  7 | # The following defines the version and build number for your application.
  8 | # A version number is three numbers separated by dots, like 1.2.43
  9 | # followed by an optional build number separated by a +.
 10 | # Both the version and the builder number may be overridden in flutter
 11 | # build by specifying --build-name and --build-number, respectively.
 12 | # In Android, build-name is used as versionName while build-number used as versionCode.
 13 | # Read more about Android versioning at https://developer.android.com/studio/publish/versioning
 14 | # In iOS, build-name is used as CFBundleShortVersionString while build-number is used as CFBundleVersion.
 15 | # Read more about iOS versioning at
 16 | #
https://developer.apple.com/library/archive/documentation/General/Reference/InfoPlistKeyReference/Articles/CoreFoundationKe
ys.html
 17 | # In Windows, build-name is used as the major, minor, and patch parts
 18 | # of the product and file versions while build-number is used as the build suffix.
 19 | version: 1.0.0+1
 20 |
 21 | environment:
 22 |   sdk: '>=3.3.3 <4.0.0'
 23 |
 24 | # Dependencies specify other packages that your package needs in order to work.
 25 | # To automatically upgrade your package dependencies to the latest versions
 26 | # consider running `flutter pub upgrade --major-versions`. Alternatively,
 27 | # dependencies can be manually updated by changing the version numbers below to
 28 | # the latest version available on pub.dev. To see which dependencies have newer
 29 | # versions available, run `flutter pub outdated`.
 30 | dependencies:
 31 |   flutter:
 32 |     sdk: flutter
```

```
33 |
34 |   lottie: ^3.1.0
35 |   cupertino_icons: ^1.0.6
36 |   firebase_core: 3.4.0
37 |   firebase_auth: 5.0.0
38 |   cloud_firestore: 5.3.0
39 |   shared_preferences: ^2.2.3
40 |   firebase_crashlytics: ^4.1.0
41 |   firebase_app_check: ^0.3.1
42 |
43 |
44 |
45 |
46 |
47 | dev_dependencies:
48 |   flutter_test:
49 |     sdk: flutter
50 |
51 |   # The "flutter_lints" package below contains a set of recommended lints to
52 |   # encourage good coding practices. The lint set provided by the package is
53 |   # activated in the `analysis_options.yaml` file located at the root of your
54 |   # package. See that file for information about deactivating specific lint
55 |   # rules and activating additional ones.
56 |   flutter_lints: ^4.0.0
57 |   flutter_launcher_icons: ^0.14.3
58 |
59 | flutter_launcher_icons:
60 |   android: true
61 |   ios: true
62 |   image_path: assets/icon/VibeLife_ikon.png
63 |
64 | # For information on the generic Dart part of this file, see the
65 | # following page: https://dart.dev/tools/pub/pubspec
66 |
67 | # The following section is specific to Flutter packages.
68 | flutter:
69 |
70 |   # The following line ensures that the Material Icons font is
71 |   # included with your application, so that you can use the icons in
72 |   # the material Icons class.
73 |   uses-material-design: true
```

Kod okurken dikkat et

dependencies: Uygulama calisirken gereken paketlerdir. Firebase paketleri giris, veritabani ve guvenlik icin kullanilir.

dev_dependencies: Sadece gelistirme ve test sirasinda gereken paketlerdir.

assets: Flutter'in uygulama icine gomdugu dosyalardir. Bu projede robot animasyonlari ve avatarlar buradan gelir.

Mini alistirma

1. firebase_auth satirini bul ve bu paketin hangi dosyalarda import edildigini ara.
2. assets/animations klasorundeki dosyalar ile robot_lottie_view.dart arasindaki baglantiyi not et.

2. Uygulamanin baslangici: main.dart

> **Bu bolumun amaci**
> main.dart uygulamanin kapisidir. Firebase baslatilir, tema yuklenir, kullanici giris yapmis mi diye bakilir ve dogru sayfaya yonlendirilir.

Bu bolumde sunlari ogren

- main() fonksiyonunun neden async oldugunu anlamak.
- runApp, MaterialApp, StatelessWidget ve StatefulWidget farkini okumak.
- StreamBuilder ve FutureBuilder ile auth akisini takip etmek.

Importlar, main() ve MaterialApp lib/main.dart:1-45

Ilk kisim uygulamanin hangi paketlere baglandigini ve ilk ekrana nasil ulastigini gosterir.

```
 1 | // Oturum durumuna göre giriş veya akış sayfası
 2 | import 'package:firebase_auth/firebase_auth.dart';
 3 | import 'package:flutter/material.dart';
 4 |
 5 | import 'app_theme.dart';
 6 | import 'auth_eposta.dart';
 7 | import 'email_dogrulama_page.dart';
 8 | import 'feed_page.dart';
 9 | import 'firebase_bootstrap.dart';
10 | import 'kayit_servis.dart';
11 | import 'login_page.dart';
12 | import 'theme_controller.dart';
13 |
14 | late final ThemeController themeController;
15 |
16 | Future<void> main() async {
17 |   WidgetsFlutterBinding.ensureInitialized();
18 |   await firebaseGuvenlikBaslat();
19 |   themeController = ThemeController();
20 |   await themeController.load();
21 |   runApp(const VibeLifeApp());
22 | }
23 |
24 | class VibeLifeApp extends StatelessWidget {
25 |   const VibeLifeApp({super.key});
26 |
27 |   @override
28 |   Widget build(BuildContext context) {
29 |     return ThemeScope(
30 |       controller: themeController,
31 |       child: ListenableBuilder(
32 |         listenable: themeController,
33 |         builder: (context, _) {
34 |           return MaterialApp(
```

```
35 |             debugShowCheckedModeBanner: false,
36 |             theme: VibeAppTheme.light,
37 |             darkTheme: VibeAppTheme.dark,
38 |             themeMode: themeController.themeMode,
39 |             home: const AuthControlPage(),
40 |           );
41 |         },
42 |       ),
43 |     );
44 |   }
45 | }
```

AuthControlPage lib/main.dart:47-124

Bu bolum kullanicinin giris durumuna gore login, e-posta dogrulama veya feed sayfasina karar verir.

```
47 | class AuthControlPage extends StatefulWidget {
48 |   const AuthControlPage({super.key});
49 |
50 |   @override
51 |   State<AuthControlPage> createState() => _AuthControlPageState();
52 | }
53 |
54 | class _AuthControlPageState extends State<AuthControlPage> {
55 |   String? _sonUid;
56 |   Future<Widget>? _sayfaFuture;
57 |
58 |   void _oturumSayfasiniYenile() {
59 |     final user = FirebaseAuth.instance.currentUser;
60 |     if (user == null) return;
61 |     setState(() {
62 |       _sonUid = user.uid;
63 |       _sayfaFuture = _girisSonrasiSayfa(user);
64 |     });
65 |   }
66 |
67 |   Future<Widget> _girisSonrasiSayfa(User user) async {
68 |     if (epostaDogrulamaBekliyorMu(user)) {
69 |       return EmailDogrulamaPage(
70 |         key: ValueKey('verify_${user.uid}'),
71 |         onDogrulandi: _oturumSayfasiniYenile,
72 |       );
73 |     }
74 |
75 |     if (await KayitServis.gecersizOturumuTemizle(user)) {
76 |       return const VibeHomePage();
77 |     }
78 |
79 |     if (!await KayitServis.profilTamamlandiMi(user.uid)) {
80 |       if (await KayitServis.bekleyenKayitVarMi(user.uid)) {
81 |         await KayitServis.dogrulamaSonrasiKaydiTamamla();
82 |       } else {
83 |         await FirebaseAuth.instance.signOut();
```

```
 84 |         return const VibeHomePage();
 85 |       }
 86 |     }
 87 |
 88 |     return FeedPage(key: ValueKey(user.uid));
 89 |   }
 90 |
 91 |   @override
 92 |   Widget build(BuildContext context) {
 93 |     return StreamBuilder<User?>(
 94 |       stream: FirebaseAuth.instance.authStateChanges(),
 95 |       builder: (context, snapshot) {
 96 |         if (snapshot.connectionState == ConnectionState.waiting &&
 97 |             FirebaseAuth.instance.currentUser == null) {
 98 |           return Scaffold(
 99 |             backgroundColor: Theme.of(context).scaffoldBackgroundColor,
100 |             body: const Center(
101 |               child: CircularProgressIndicator(color: VibeAppTheme.accent),
102 |             ),
103 |           );
104 |         }
105 |
106 |         final user = snapshot.data ?? FirebaseAuth.instance.currentUser;
107 |         if (user == null) {
108 |           _sonUid = null;
109 |           _sayfaFuture = null;
110 |           return const VibeHomePage();
111 |         }
112 |
113 |         if (_sonUid != user.uid || _sayfaFuture == null) {
114 |           _sonUid = user.uid;
115 |           _sayfaFuture = _girisSonrasiSayfa(user);
116 |         }
117 |
118 |         return FutureBuilder<Widget>(
119 |           future: _sayfaFuture,
120 |           builder: (context, sayfaSnap) {
121 |             if (sayfaSnap.connectionState != ConnectionState.done) {
122 |               return Scaffold(
123 |                 backgroundColor: Theme.of(context).scaffoldBackgroundColor,
124 |                 body: const Center(
```

Kod okurken dikkat et

import: Baska dosyadaki sinif ve fonksiyonlari bu dosyada kullanmak icin iceri alir.

late final: Deger sonradan atanacak ama atandiktan sonra degismeyecek demektir.

async / await: Firebase ve yerel depolama gibi bekleme gerektiren islemleri sirali ve okunabilir yapar.

StreamBuilder: Auth durumu degistikce ekrani yeniden kurar. Kullanici giris yaparsa stream yeni User degeri verir.

FutureBuilder: Bir defalik asenkron karar bekler: e-posta dogrulamasi, profil tamamlandi mi gibi.

?: Degerin null olabilecegini soyler. Ornek: String? _sonUid.

!: Kod yazari burada null olmadigindan emin oldugunu soyluyor. Yanlis kullanilirsa hata uretir.

Mini alistirma

1. main.dart icinde FirebaseAuth.instance gecen satirlari isaretle.
2. VibeHomePage ne zaman donuyor, FeedPage ne zaman donuyor kendi cumlenle yaz.
3. AuthControlPage'i bir trafik polisi gibi dusun: hangi kosulda hangi ekrana gonderiyor?

3. Firebase baslatma ve guvenlik

> **Bu bolumun amaci**
> Firebase uygulamanin kimlik, veritabani, crash raporu ve App Check tarafini tasir. Bu dosya uygulama acilirken once calisir.

Bu bolumde sunlari ogren

- Firebase.initializeApp ne zaman calisir anlamak.
- Crashlytics ve App Check'in neden web disinda kosullu calistigini gormek.
- switch-case yapisini ve platform kontrolunu okumak.

Firebase bootstrap lib/firebase_bootstrap.dart:1-60

Bu dosya main.dart icinden cagrilir ve Firebase servislerini hazirlar.

```
// Firebase App Check ve Crashlytics başlatma
import 'package:firebase_app_check/firebase_app_check.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_crashlytics/firebase_crashlytics.dart';
import 'package:flutter/foundation.dart';

import 'app_check_config.dart';
import 'firebase_options.dart';

// Firebase çekirdek + güvenlik servislerini başlatır
Future<void> firebaseGuvenlikBaslat() async {
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  await _crashlyticsBaslat();
  await _appCheckBaslat();
}

Future<void> _crashlyticsBaslat() async {
  if (kIsWeb) return;

  final desteklenen = defaultTargetPlatform == TargetPlatform.android ||
      defaultTargetPlatform == TargetPlatform.iOS ||
      defaultTargetPlatform == TargetPlatform.macOS;
  if (!desteklenen) return;

  await FirebaseCrashlytics.instance
      .setCrashlyticsCollectionEnabled(!kDebugMode);

  FlutterError.onError =
      FirebaseCrashlytics.instance.recordFlutterFatalError;

  PlatformDispatcher.instance.onError = (error, stack) {
```

```
35 |     FirebaseCrashlytics.instance.recordError(error, stack, fatal: true);
36 |     return true;
37 |   };
38 | }
39 |
40 | Future<void> _appCheckBaslat() async {
41 |   if (kIsWeb) return;
42 |
43 |   const betaVeyaDebug = appCheckBetaModu || kDebugMode;
44 |
45 |   switch (defaultTargetPlatform) {
46 |     case TargetPlatform.android:
47 |       await FirebaseAppCheck.instance.activate(
48 |         androidProvider: betaVeyaDebug
49 |             ? AndroidProvider.debug
50 |             : AndroidProvider.playIntegrity,
51 |       );
52 |       break;
53 |     case TargetPlatform.iOS:
54 |     case TargetPlatform.macOS:
55 |       await FirebaseAppCheck.instance.activate(
56 |         appleProvider: betaVeyaDebug
57 |             ? AppleProvider.debug
58 |             : AppleProvider.appAttest,
59 |       );
60 |       break;
```

Kod okurken dikkat et

kIsWeb: Kodun web platformunda calisip calismadigini soyler.

defaultTargetPlatform: Android, iOS, macOS gibi platforma gore farkli ayar yapmaya yarar.

try / catch: Hata olursa uygulamanin tamamen cokmesini engelleyip hatayi kontrollu yakalar.

Mini alistirma

1. App Check debug modda hangi provider'i kullaniyor bul.
2. Crashlytics hangi platformlarda aktif ediliyor not et.

4. Kucuk dosyalardan Dart ogrenme

> **Bu bolumun amaci**
> Buyuk ekran dosyalarindan once kucuk yardimci dosyalarla Dart dilinin temel isaretlerini ve class mantigini ogrenmek daha rahattir.

Bu bolumde sunlari ogren

- enum, class, getter, final ve static kavramlarini okumak.
- SharedPreferences ile cihaz icine kucuk bilgi kaydetmeyi anlamak.
- Snackbar gibi ortak bir UI yardimcisinin nasil tasarlandigini gormek.

Tema kontrolu lib/theme_controller.dart:1-47

Tema tercihi hafizada tutulur ve degisince ekranlara haber verilir.

```
// Tema tercihini yönetir ve kaydeder.
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

// Uygulama görünüm seçenekleri
enum VibeAppearance { dark, light }

// Tema durumunu tutan ve dinleyicilere bildiren sınıf
class ThemeController extends ChangeNotifier {
  static const _storageKey = 'vibe_appearance'; // Yerel depolama anahtarı

  VibeAppearance _appearance = VibeAppearance.dark; // Varsayılan koyu tema

  VibeAppearance get appearance => _appearance;

  ThemeMode get themeMode =>
      _appearance == VibeAppearance.dark ? ThemeMode.dark : ThemeMode.light;

  bool get isDark => _appearance == VibeAppearance.dark;

  // Kayıtlı temayı SharedPreferences'tan okur
  Future<void> load() async {
    final prefs = await SharedPreferences.getInstance();
    final saved = prefs.getString(_storageKey);
    if (saved == VibeAppearance.light.name) {
      _appearance = VibeAppearance.light;
    } else {
      _appearance = VibeAppearance.dark;
    }
    notifyListeners();
  }

  // Temayı değiştirir ve kalıcı olarak kaydeder
  Future<void> setAppearance(VibeAppearance value) async {
```

```
    if (_appearance == value) return;
    _appearance = value;
    notifyListeners();
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_storageKey, value.name);
  }
}

// Alt widget'lara ThemeController erişimi sağlar
class ThemeScope extends InheritedNotifier<ThemeController> {
  const ThemeScope({
    super.key,
    required ThemeController controller,
```

Ortak snackbar lib/vibe_snackbar.dart:1-92

Uygulamanin hata, uyari, bilgi ve basari mesajlari tek yerden stillendirilir.

```
// Özelleştirilmiş snackbar mesajları.
import 'package:flutter/material.dart';
import 'package:vibe_life/app_theme.dart';

// Snackbar mesaj türleri
enum VibeMesajTipi { hata, uyari, bilgi, basari }

// Tipe göre stillendirilmiş snackbar gösterir
void vibeSnackbarGoster(
  BuildContext context,
  String mesaj, {
  VibeMesajTipi tip = VibeMesajTipi.hata,
}) {
  final theme = Theme.of(context);
  final isDark = theme.brightness == Brightness.dark;

  late final Color arkaPlan;
  late final Color yaziRengi;
  late final Color? kenarlik;
  late final double radius;
  late final IconData ikon;

  // Tipe göre renk ve ikon seç
  switch (tip) {
    case VibeMesajTipi.hata:
      arkaPlan = const Color.fromARGB(255, 120, 0, 0);
      yaziRengi = Colors.white;
      kenarlik = null;
      radius = 10;
      ikon = Icons.error_outline_rounded;
      break;
    case VibeMesajTipi.uyari:
      arkaPlan = isDark ? const Color(0xFF4A3800) : const Color(0xFFFFF3E0);
      yaziRengi = isDark ? const Color(0xFFFFE082) : const Color(0xFF8D4B00);
      kenarlik = isDark ? const Color(0xFFFFB74D) : const Color(0xFFFFCC80);
```

```
      radius = 18;
      ikon = Icons.warning_amber_rounded;
      break;
    case VibeMesajTipi.bilgi:
      arkaPlan = isDark ? const Color(0xFF2A1A3E) : const Color(0xFFF3EBF8);
      yaziRengi = isDark ? Colors.white : const Color(0xFF3D2A4F);
      kenarlik = isDark ? VibeAppTheme.accent.withOpacity(0.45) : null;
      radius = 14;
      ikon = Icons.info_outline_rounded;
      break;
    case VibeMesajTipi.basari:
      arkaPlan = isDark ? const Color(0xFF1B4D3E) : const Color(0xFFE8F5E9);
      yaziRengi = isDark ? const Color(0xFFA5D6A7) : const Color(0xFF1B5E20);
      kenarlik = isDark ? const Color(0xFF66BB6A) : null;
      radius = 14;
      ikon = Icons.check_circle_outline_rounded;
      break;
  }

  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      behavior: SnackBarBehavior.floating,
      backgroundColor: Colors.transparent,
      elevation: 0,
      margin: const EdgeInsets.fromLTRB(16, 0, 16, 16),
      content: Container(
        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
        decoration: BoxDecoration(
          color: arkaPlan,
          borderRadius: BorderRadius.circular(radius),
          border: kenarlik != null
              ? Border.all(color: kenarlik, width: 1.2)
              : null,
          boxShadow: [
            BoxShadow(
              color: Colors.black.withOpacity(isDark ? 0.35 : 0.12),
              blurRadius: 12,
              offset: const Offset(0, 4),
            ),
          ],
        ),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Icon(ikon, color: yaziRengi, size: 20),
            const SizedBox(width: 10),
            Expanded(
              child: Text(
                mesaj,
                style: TextStyle(
                  color: yaziRengi,
                  fontSize: 14,
                  height: 1.35,
```

```
89 |                ),
90 |              ),
91 |            ),
92 |          ],
```

Tarih formatlama lib/utils/tarih_saat.dart:1-13

Firestore Timestamp degerini ekranda okunur metne ceviren kucuk ve temiz fonksiyon.

```
1 | // Firestore Timestamp değerlerini okunabilir tarih metnine çevirir
2 | import 'package:cloud_firestore/cloud_firestore.dart';
3 |
4 | // Timestamp'i gg.aa.yyyy ss:dd formatına dönüştürür
5 | String tarihSaatYaz(dynamic value) {
6 |   if (value is! Timestamp) return "Tarih yok"; // Geçersiz değer kontrolü
7 |
8 |   final tarih = value.toDate(); // DateTime'a çevir
9 |   final gun = tarih.day.toString().padLeft(2, "0");
10 |   final ay = tarih.month.toString().padLeft(2, "0");
11 |   final yil = tarih.year.toString();
12 |   final saat = tarih.hour.toString().padLeft(2, "0");
13 |   final dakika = tarih.minute.toString().padLeft(2, "0");
```

Kod okurken dikkat et

enum: Sinirli sayida sabit secenek tanimlar. Ornek: VibeMesajTipi.hata veya VibeAppearance.dark.

ChangeNotifier: Bir deger degistiginde onu dinleyen widget'lara haber vermek icin kullanilir.

get: Fonksiyon gibi hesaplanan ama alan gibi okunan degerdir. Ornek: themeMode.

switch: Bir degere gore farkli kod bloklari calistirir.

BuildContext: Widget'in ekrandaki yerini ve tema/navigator gibi cevre bilgilerini tasir.

Mini alistirma

1. VibeMesajTipi enumuna yeni bir tip eklense switch icinde neden yeni case gerekir?
2. theme_controller.dart icinde notifyListeners hangi durumlarda cagiriliyor bul.

5. Ana kabuk ve alt sekmeler: feed_page.dart

> **Bu bolumun amaci**
> Kullanici giris yaptiktan sonra uygulamanin ana haritasi bu dosyada kurulur. Feed, kesfet, yeni gonderi, bildirim ve profil sekmeleri buradan acilir.

Bu bolumde sunlari ogren

- StatefulWidget'in yasam dongusunu okumak.
- initState, dispose ve didChangeAppLifecycleState ne is yapar anlamak.
- IndexedStack ve BottomNavigationBar ile sayfa degistirmeyi takip etmek.

Sekme sistemi ve yasam dongusu lib/feed_page.dart:1-95

Bu kisim hangi sekmede hangi sayfanin acilacagini ve ilk acilista hangi servislerin calistigini gosterir.

```
 1 | // Ana sayfa kabuğu: alt menü ile feed, keşfet, yeni gönderi, bildirim ve profil sekmeleri
 2 | import 'dart:async';
 3 |
 4 | import 'package:firebase_auth/firebase_auth.dart';
 5 | import 'package:flutter/material.dart';
 6 | import 'package:shared_preferences/shared_preferences.dart';
 7 | import 'package:vibe_life/explore_page.dart';
 8 | import 'package:vibe_life/feed/feed_content.dart';
 9 | import 'package:vibe_life/haftalik_gorev_servis.dart';
10 | import 'package:vibe_life/legal_pages.dart';
11 | import 'package:vibe_life/newpost_page.dart';
12 | import 'package:vibe_life/notifications_page.dart';
13 | import 'package:vibe_life/profile_page.dart';
14 | import 'package:vibe_life/robot_servis.dart';
15 | import 'package:vibe_life/sosyal_servis.dart';
16 | import 'package:vibe_life/vibe_snackbar.dart';
17 | import 'package:vibe_life/widgets/vibe_gradient_scaffold.dart';
18 |
19 | // Feed modülü bileşenlerini dışa aktar
20 | export 'package:vibe_life/feed/feed_content.dart';
21 | export 'package:vibe_life/widgets/post_card.dart';
22 | export 'package:vibe_life/widgets/yorum_paneli.dart';
23 |
24 | // Ana uygulama sayfası (StatefulWidget)
25 | class FeedPage extends StatefulWidget {
26 |   const FeedPage({super.key});
27 |
28 |   @override
29 |   State<FeedPage> createState() => _FeedPageState();
30 | }
31 |
32 | // Feed sayfası durumu ve yaşam döngüsü yönetimi
33 | class _FeedPageState extends State<FeedPage> with WidgetsBindingObserver {
34 |   int _secilenIndeks = 0;
```

```
  Timer? _robotDakikaTimer;
  final _feedKey = GlobalKey<FeedPageContentState>();
  final Map<int, Widget> _sayfaOnbellek = {};
  final Set<int> _acilanSekmeler = {0};

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    _robotSenkronBaslat();
    _robotDakikaTimerBaslat();
    _surumNotuGoster();
    _gunlukGorevAcilisKaydet();
  }

  void _sekmeyeGit(int indeks) {
    setState(() {
      _acilanSekmeler.add(indeks);
      _secilenIndeks = indeks;
    });
  }

  Widget _sayfaOlustur(int indeks) {
    switch (indeks) {
      case 0:
        return FeedPageContent(
          key: _feedKey,
          onKendiProfilineGit: () => _sekmeyeGit(4),
        );
      case 1:
        return const ExplorePage();
      case 2:
        return NewPostPage(
          onPostCreated: () {
            FocusManager.instance.primaryFocus?.unfocus();
            _sekmeyeGit(0);
            _feedKey.currentState?.feediYenile();
          },
        );
      case 3:
        return const NotificationsPage();
      case 4:
        return const ProfilePage();
      default:
        return const SizedBox.shrink();
    }
  }

  Widget _sayfaWidget(int indeks) {
    return _sayfaOnbellek.putIfAbsent(indeks, () => _sayfaOlustur(indeks));
  }

  List<Widget> get _sekmeGovdeleri => List.generate(
```

```
 88 |         5,
 89 |         (indeks) => _acilanSekmeler.contains(indeks)
 90 |             ? _sayfaWidget(indeks)
 91 |             : const SizedBox.shrink(),
 92 |       );
 93 |
 94 |   // Oturum açılışında görev ve takip ödüllerini senkronize et
 95 |   void _gunlukGorevAcilisKaydet() {
```

Robot senkronu ve surum notu lib/feed_page.dart:97-168

Uygulama acilinca veya on plana gelince robot ve gorev senkronizasyonu yapilir.

```
 97 |     if (uid == null) return;
 98 |     HaftalikGorevServis.oturumSenkronizeEt(uid);
 99 |     unawaited(SosyalServis.takipOdulleriniSenkronizeEt(uid));
100 |   }
101 |
102 |   @override
103 |   void dispose() {
104 |     WidgetsBinding.instance.removeObserver(this);
105 |     _robotDakikaTimer?.cancel();
106 |     super.dispose();
107 |   }
108 |
109 |   // Uygulama ön/arka plana geçince robot ve görevleri yönet
110 |   @override
111 |   void didChangeAppLifecycleState(AppLifecycleState state) {
112 |     if (state == AppLifecycleState.resumed) {
113 |       _robotSenkronBaslat();
114 |       _robotDakikaTimerBaslat();
115 |       _gunlukGorevAcilisKaydet();
116 |     } else if (state == AppLifecycleState.paused ||
117 |         state == AppLifecycleState.inactive) {
118 |       _robotDakikaTimer?.cancel(); // Arka planda zamanlayıcıyı durdur
119 |     }
120 |   }
121 |
122 |   // Her dakika robot dakika senkronunu çalıştır
123 |   void _robotDakikaTimerBaslat() {
124 |     _robotDakikaTimer?.cancel();
125 |     _robotDakikaTimer = Timer.periodic(const Duration(minutes: 5), (_) {
126 |       final uid = FirebaseAuth.instance.currentUser?.uid;
127 |       if (uid != null) {
128 |         RobotServis.dakikaSenkronizeEt(uid);
129 |       }
130 |     });
131 |   }
132 |
133 |   // Robot senkron mesajlarını snackbar ile göster
134 |   void _robotSenkronBaslat() {
135 |     final uid = FirebaseAuth.instance.currentUser?.uid;
136 |     if (uid == null) return;
```

```
137 |     RobotServis.senkronizeEt(uid).then((sonuc) {
138 |       if (!mounted || sonuc == null || sonuc.mesajlar.isEmpty) return;
139 |       final messenger = ScaffoldMessenger.maybeOf(context);
140 |       if (messenger == null) return;
141 |       for (final mesaj in sonuc.mesajlar) {
142 |         // Mesaj tipine göre snackbar rengi seç
143 |         final tip = RobotServis.senkronMesajBasariMi(mesaj)
144 |             ? VibeMesajTipi.basari
145 |             : RobotServis.senkronMesajBilgiMi(mesaj)
146 |                 ? VibeMesajTipi.bilgi
147 |                 : VibeMesajTipi.uyari;
148 |         vibeSnackbarGoster(context, mesaj, tip: tip);
149 |       }
150 |     });
151 |   }
152 |
153 |   // İlk açılışta sürüm yeniliklerini bir kez göster
154 |   Future<void> _surumNotuGoster() async {
155 |     final prefs = await SharedPreferences.getInstance();
156 |     const key = 'last_seen_changelog_version';
157 |     final seen = prefs.getString(key);
158 |     if (seen == VibeLegal.appVersionLabel) return; // Zaten görüldü
159 |     if (!mounted) return;
160 |     await Future<void>.delayed(const Duration(milliseconds: 450));
161 |     if (!mounted) return;
162 |     await showDialog<void>(
163 |       context: context,
164 |       builder: (context) => AlertDialog(
165 |         title: const Text('Yenilikler v${VibeLegal.appVersionLabel}'),
166 |         content: const Text(
167 |           '- Robot sistemi iyilestirildi\n'
168 |           '- Gizlilik ve guvenlik ayarlari guclendirildi\n'
```

Alt navigasyon lib/feed_page.dart:178-257

BottomNavigationBar kullanicinin sekmeler arasinda gezmesini saglar; bildirim sekmesinde okunmamis sayac vardir.

```
178 |     );
179 |     await prefs.setString(key, VibeLegal.appVersionLabel); // Görüldü olarak kaydet
180 |   }
181 |
182 |   @override
183 |   Widget build(BuildContext context) {
184 |     final theme = Theme.of(context);
185 |
186 |     return VibeGradientScaffold(
187 |       extendBodyBehindAppBar: false,
188 |       appBar: AppBar(
189 |         title: const Text(
190 |           "VibeLife",
191 |           style: TextStyle(
192 |             fontWeight: FontWeight.bold,
```

```
193 |             letterSpacing: 1.2,
194 |           ),
195 |         ),
196 |       ),
197 |       // Seçili sekme görünür, diğerleri bellekte tutulur
198 |       body: IndexedStack(
199 |         index: _secilenIndeks,
200 |         children: _sekmeGovdeleri,
201 |       ),
202 |       bottomNavigationBar: BottomNavigationBar(
203 |         type: BottomNavigationBarType.fixed,
204 |         backgroundColor: theme.bottomNavigationBarTheme.backgroundColor,
205 |         selectedItemColor: theme.bottomNavigationBarTheme.selectedItemColor,
206 |         unselectedItemColor: theme.bottomNavigationBarTheme.unselectedItemColor,
207 |         showSelectedLabels: false,
208 |         showUnselectedLabels: false,
209 |         currentIndex: _secilenIndeks,
210 |         onTap: (indeks) {
211 |           FocusManager.instance.primaryFocus?.unfocus();
212 |           _sekmeyeGit(indeks);
213 |         },
214 |         items: [
215 |           const BottomNavigationBarItem(icon: Icon(Icons.home), label: "Ana"),
216 |           const BottomNavigationBarItem(
217 |               icon: Icon(Icons.search), label: "Keşfet"),
218 |           const BottomNavigationBarItem(
219 |             icon: Icon(Icons.add_circle_outline),
220 |             label: "Yeni",
221 |           ),
222 |           // Okunmamış bildirim rozeti
223 |           BottomNavigationBarItem(
224 |             icon: StreamBuilder<int>(
225 |               stream: FirebaseAuth.instance.currentUser == null
226 |                   ? Stream.value(0)
227 |                   : SosyalServis.okunmamisBildirimSayisi(
228 |                       FirebaseAuth.instance.currentUser!.uid,
229 |                     ),
230 |               builder: (context, snapshot) {
231 |                 final sayi = snapshot.data ?? 0;
232 |                 return Stack(
233 |                   clipBehavior: Clip.none,
234 |                   children: [
235 |                     const Icon(Icons.notifications_none),
236 |                     if (sayi > 0)
237 |                       Positioned(
238 |                         right: -6,
239 |                         top: -4,
240 |                         child: Container(
241 |                           padding: const EdgeInsets.symmetric(
242 |                             horizontal: 5,
243 |                             vertical: 1,
244 |                           ),
245 |                           constraints: const BoxConstraints(minWidth: 18),
```

```
246 |                         decoration: BoxDecoration(
247 |                           color: Colors.redAccent,
248 |                           borderRadius: BorderRadius.circular(10),
249 |                         ),
250 |                         child: Text(
251 |                           sayi > 99 ? "99+" : "$sayi",
252 |                           textAlign: TextAlign.center,
253 |                           style: const TextStyle(
254 |                             color: Colors.white,
255 |                             fontSize: 10,
256 |                             fontWeight: FontWeight.bold,
257 |                           ),
```

Kod okurken dikkat et

initState: Sayfa ilk olustugunda bir kez calisir.

dispose: Sayfa kapanirken timer gibi kaynaklari temizler.

Timer.periodic: Belirli araliklarla ayni islemi tekrar eder.

IndexedStack: Secili sekmeyi gosterirken diger acilmis sekmeleri bellekte tutar.

setState: State icindeki deger degisti, ekrani yeniden ciz demektir.

Mini alistirma

1. _secilenIndeks degeri 0 iken hangi sayfa gorunur?
2. Yeni gonderi paylasildiktan sonra neden _sekmeyeGit(0) cagriliyor?

6. Gonderi paylasma akisi

> **Bu bolumun amaci**
> Bir butona basildiginda UI, Firebase Auth, Firestore batch, sosyal servis, robot servisi ve gorev sistemi birlikte calisir. Bu bolum uygulamanin en ogretici akislarindan biridir.

Bu bolumde sunlari ogren

- Form verisini okumak ve bosluk kontrolu yapmak.
- Firestore batch ile birden fazla yazmayi birlikte yapmak.
- try/catch/finally ile yukleniyor durumunu dogru kapatmak.

Sayfa ve controller lib/newpost_page.dart:1-40

TextEditingController kullanicinin yazdigi metni okur; isLoading buton durumunu kontrol eder.

```
 1 | // Yeni gönderi oluşturma sayfası.
 2 | import 'dart:async';
 3 |
 4 | import 'package:cloud_firestore/cloud_firestore.dart';
 5 | import 'package:firebase_auth/firebase_auth.dart';
 6 | import 'package:flutter/material.dart';
 7 | import 'package:vibe_life/app_theme.dart';
 8 | import 'package:vibe_life/error_messages.dart';
 9 | import 'package:vibe_life/haftalik_gorev_servis.dart';
10 | import 'package:vibe_life/puan_animasyon.dart';
11 | import 'package:vibe_life/robot_servis.dart';
12 | import 'package:vibe_life/sosyal/sosyal_core.dart';
13 | import 'package:vibe_life/sosyal_servis.dart';
14 | import 'package:vibe_life/widgets/bahsetme_metin_alani.dart';
15 | import 'package:vibe_life/vibe_snackbar.dart';
16 | import 'package:vibe_life/widgets/robot_lottie_view.dart';
17 |
18 | // Gönderi paylaşım formu
19 | class NewPostPage extends StatefulWidget {
20 |   final VoidCallback? onPostCreated; // Paylaşım sonrası callback
21 |
22 |   const NewPostPage({super.key, this.onPostCreated});
23 |
24 |   @override
25 |   State<NewPostPage> createState() => _NewPostPageState();
26 | }
27 |
28 | class _NewPostPageState extends State<NewPostPage> {
29 |   final TextEditingController postController = TextEditingController(); // Gönderi metni
30 |   bool isLoading = false; // Paylaşım sürüyor mu
31 |
32 |   @override
33 |   void dispose() {
34 |     postController.dispose();
```

```
35 |     super.dispose();
36 |   }
37 |
38 |   // Gönderiyi Firestore'a kaydeder ve puan/görev günceller
39 |   Future<void> gonderiPaylas() async {
40 |     final metin = postController.text.trim();
```

gonderiPaylas fonksiyonu lib/newpost_page.dart:42-164

Gonderi paylasmanin ana is akisi: kontrol, Firestore yazma, puan, mention, gorev, robot aksiyonu.

```
42 |
43 |     if (user == null) {
44 |       mesajGoster("Kullanıcı bulunamadı. Tekrar giriş yap.");
45 |       return;
46 |     }
47 |
48 |     if (metin.isEmpty) {
49 |       mesajGoster("Paylaşmak için bir şeyler yaz.");
50 |       return;
51 |     }
52 |
53 |     if (metin.length > SosyalCore.gonderiMetinMaxKarakter) {
54 |       mesajGoster(
55 |         "Gönderi en fazla ${SosyalCore.gonderiMetinMaxKarakter} karakter olabilir.",
56 |       );
57 |       return;
58 |     }
59 |
60 |     setState(() {
61 |       isLoading = true;
62 |     });
63 |
64 |     try {
65 |       await SosyalServis.gonderiPaylasimHizLimitiKontrol(user.uid);
66 |       await RobotServis.etkilesimEngelliyseAt(
67 |         user.uid,
68 |         aksiyonTipi: 'gonderi',
69 |       );
70 |
71 |       final userDoc = await FirebaseFirestore.instance
72 |           .collection("users")
73 |           .doc(user.uid)
74 |           .get();
75 |
76 |       final userData = userDoc.data();
77 |       final kullaniciAdi =
78 |           userData?["kullaniciAdi"] ?? user.displayName ?? "Vibe Kullanıcısı";
79 |       final photoUrl =
80 |           (userData?["photoUrl"] ?? user.photoURL ?? "").toString();
81 |       final avatarKey = (userData?["avatarKey"] ?? "").toString();
82 |
83 |       final hesapGizli = userData?["hesapGizli"] == true;
```

```
84 |       final mentionUidleri = await SosyalServis.bahsetmeUidleriniCoz(metin);
85 |
86 |       final firestore = FirebaseFirestore.instance;
87 |       final postRef = firestore.collection("posts").doc();
88 |       final batch = firestore.batch();
89 |
90 |       batch.set(postRef, {
91 |         "userId": user.uid,
92 |         "kullaniciAdi": kullaniciAdi,
93 |         "photoUrl": photoUrl,
94 |         "avatarKey": avatarKey,
95 |         "metin": metin,
96 |         "tags": SosyalServis.etiketleriAyikla(metin),
97 |         "bahsetmeler": SosyalServis.bahsetmeleriAyikla(metin),
98 |         "mentionUidleri": mentionUidleri,
99 |         "begeniSayisi": 0,
100 |        "yorumSayisi": 0,
101 |        "skor": 0,
102 |        "sikayetSayisi": 0,
103 |        "raporNedeniyleGizli": false,
104 |        "ownerHesapGizli": hesapGizli,
105 |        "createdAt": FieldValue.serverTimestamp(),
106 |      });
107 |      batch.update(firestore.collection("users").doc(user.uid), {
108 |        "puan": FieldValue.increment(10),
109 |        "gonderiSayisi": FieldValue.increment(1),
110 |        "sonGonderiPuanPostId": postRef.id,
111 |      });
112 |
113 |      try {
114 |        await batch.commit();
115 |      } on FirebaseException catch (e) {
116 |        mesajGoster(_firestoreHataMesaji(e));
117 |        return;
118 |      }
119 |
120 |      await SosyalServis.gonderiPaylasimHizLimitiKaydet(user.uid);
121 |
122 |      if (mentionUidleri.isNotEmpty) {
123 |        await SosyalServis.bahsetmeBildirimleriGonder(
124 |          gonderenUid: user.uid,
125 |          mentionUidleri: mentionUidleri,
126 |          kaynak: "post",
127 |          postId: postRef.id,
128 |        );
129 |      }
130 |
131 |      final gorevSonuc = await HaftalikGorevServis.aksiyonKaydet(
132 |        user.uid,
133 |        GorevAksiyon.gonderi,
134 |      );
135 |      await RobotServis.sosyalAksiyon(user.uid, 'gonderi');
136 |
```

```
137 |       postController.clear();
138 |
139 |       if (!mounted) return;
140 |
141 |       puanAnimasyonuGoster(context, 10);
142 |       if (gorevSonuc.basarili) {
143 |         for (final mesaj in gorevSonuc.mesajlar) {
144 |           mesajGoster(mesaj, tip: VibeMesajTipi.basari);
145 |         }
146 |       } else if (gorevSonuc.hata != null &&
147 |           gorevSonuc.hata != 'Görevler 5. seviyede açılır.') {
148 |         mesajGoster(gorevSonuc.hata!, tip: VibeMesajTipi.uyari);
149 |       }
150 |       FocusManager.instance.primaryFocus?.unfocus();
151 |       widget.onPostCreated?.call();
152 |     } catch (e) {
153 |       mesajGoster(
154 |         internetAwareErrorMessage(
155 |           error: e,
156 |           fallback: "Gönderi paylaşılamadı.",
157 |         ),
158 |       );
159 |     } finally {
160 |       if (mounted) {
161 |         setState(() {
162 |           isLoading = false;
163 |         });
164 |       }
```

Kod okurken dikkat et

TextEditingController: TextField icindeki metni kod tarafindan okumaya ve temizlemeye yarar.

trim(): Metnin basindaki ve sonundaki bosluklari siler.

batch.set / batch.update: Birden fazla Firestore islemini tek commit ile yapar.

FieldValue.increment: Sayaclari mevcut degerin ustune ekleyerek artirir.

finally: Islem basarili da olsa hata da olsa calisir; loading kapatmak icin idealdir.

Mini alistirma

1. Kullanici null ise neden Firestore'a hic gidilmiyor?
2. Post olusturulurken posts koleksiyonuna hangi alanlar yaziliyor tek tek listele.
3. RobotServis.sosyalAksiyon satiri silinse oyun mekaniginde ne eksilir?

7. Sosyal servis katmani

> **Bu bolumun amaci**
> UI dosyalari Firestore detaylarini her yerde tekrar etmek yerine SosyalServis ve alt servisleri kullanir. Bu, kodun daha okunabilir ve toparli kalmasini saglar.

Bu bolumde sunlari ogren

- Facade desenini bu projedeki haliyle gormek.
- SosyalCore icindeki ortak hiz limiti ve engel kontrolunu anlamak.
- Begeni, yorum, takip ve mesaj fonksiyonlarinin tek kapidan gecmesini okumak.

Ortak yardimcilar ve hiz limiti lib/sosyal/sosyal_core.dart:1-90

Sosyal aksiyonlar icin ortak Firestore ornegi, limitler ve rate-limit kontrolu burada bulunur.

```
 1 | // Sosyal modül ortak yardımcıları, limitler ve Firestore işlemleri
 2 | import 'package:cloud_firestore/cloud_firestore.dart';
 3 | import 'package:firebase_auth/firebase_auth.dart';
 4 | import 'package:flutter/foundation.dart';
 5 | import 'package:shared_preferences/shared_preferences.dart';
 6 | import 'package:vibe_life/sosyal/bildirim_servis.dart';
 7 |
 8 | // Ortak yardımcı metotlar ve sabitler
 9 | class SosyalCore {
10 |   static final db = FirebaseFirestore.instance; // Firestore veritabanı örneği
11 |   static const int postRaporEsigi = 5; // Gönderi gizleme şikayet eşiği
12 |   static const int yorumRaporEsigi = 4; // Yorum gizleme şikayet eşiği
13 |   static const int yorumMinAralikMs = 4000; // Yorumlar arası min süre
14 |   static const int begeniMinAralikMs = 1200; // Beğeniler arası min süre
15 |   static const int gonderiMinAralikMs = 6000; // Gönderiler arası min süre
16 |   /// Firestore doküman sınırına uygun pratik üst limit (UI'da sınırsız hissi).
17 |   static const int gonderiMetinMaxKarakter = 10000;
18 |   static const int whereInLimit = 10; // Firestore whereIn parça boyutu
19 |   static final Map<String, int> _sonAksiyonMs = {}; // Bellekte son aksiyon zamanı
20 |   static SharedPreferences? _rateLimitPrefs; // Kalıcı hız limiti tercihleri
21 |
22 |   // SharedPreferences örneğini yükler
23 |   static Future<void> rateLimitPrefsYukle() async {
24 |     _rateLimitPrefs ??= await SharedPreferences.getInstance();
25 |   }
26 |
27 |   // Aksiyon hız limitini kontrol eder
28 |   static Future<void> aksiyonHizLimitiKontrol({
29 |     required String uid,
30 |     required String aksiyon,
31 |     required int minAralikMs,
32 |     required String mesaj,
33 |   }) async {
34 |     await rateLimitPrefsYukle();
```

```
    final key = '$uid:$aksiyon'; // Kullanıcı ve aksiyon anahtarı
    final prefsKey = 'rate_limit:$key';
    final simdi = DateTime.now().millisecondsSinceEpoch;
    final bellekten = _sonAksiyonMs[key] ?? 0;
    final kayitli = _rateLimitPrefs!.getInt(prefsKey) ?? 0;
    final son = bellekten > kayitli ? bellekten : kayitli; // En güncel zamanı al
    if (son > 0 && simdi - son < minAralikMs) {
      throw mesaj; // Çok hızlıysa hata fırlat
    }
  }

  // Son aksiyon zamanını kaydeder
  static Future<void> aksiyonHizLimitiKaydet({
    required String uid,
    required String aksiyon,
  }) async {
    await rateLimitPrefsYukle();
    final key = '$uid:$aksiyon';
    final prefsKey = 'rate_limit:$key';
    final simdi = DateTime.now().millisecondsSinceEpoch;
    _sonAksiyonMs[key] = simdi; // Belleğe yaz
    await _rateLimitPrefs!.setInt(prefsKey, simdi); // Kalıcı depoya yaz
  }

  // Gönderi paylaşım hız limitini kontrol eder
  static Future<void> gonderiPaylasimHizLimitiKontrol(String uid) {
    return aksiyonHizLimitiKontrol(
      uid: uid,
      aksiyon: 'post',
      minAralikMs: gonderiMinAralikMs,
      mesaj: 'Cok hizli gonderi paylasiyorsun. Lutfen biraz bekle.',
    );
  }

  // Gönderi paylaşım zamanını kaydeder
  static Future<void> gonderiPaylasimHizLimitiKaydet(String uid) {
    return aksiyonHizLimitiKaydet(uid: uid, aksiyon: 'post');
  }

  // İki kullanıcı arasında engel var mı
  static Future<bool> engellendiMi({
    required String kaynakUid,
    required String hedefUid,
  }) async {
    try {
      // Kaynak kullanıcının engellediklerini kontrol et
      final blocked = await db
          .collection("users")
          .doc(kaynakUid)
          .collection("blocked")
          .doc(hedefUid)
          .get();
      if (blocked.exists) return true;
```

```
88 |
89 |       // Kaynak kullanıcıyı engelleyenleri kontrol et
90 |       final blockedBy = await db
```

Facade baslangici lib/sosyal_servis.dart:1-60

SosyalServis UI tarafina tek bir sinif gibi gorunur; gercekte isi alt servislere dagitir.

```
 1 | // Sosyal alt servislerin tek noktadan erişim cephesi
 2 | import 'package:cloud_firestore/cloud_firestore.dart';
 3 | import 'package:vibe_life/sosyal/begeni_yorum_servis.dart';
 4 | import 'package:vibe_life/sosyal/bildirim_servis.dart';
 5 | import 'package:vibe_life/sosyal/engel_servis.dart';
 6 | import 'package:vibe_life/sosyal/etiket_servis.dart';
 7 | import 'package:vibe_life/sosyal/mesaj_servis.dart';
 8 | import 'package:vibe_life/sosyal/sosyal_core.dart';
 9 | import 'package:vibe_life/sosyal/takip_servis.dart';
10 |
11 | export 'sosyal/takip_servis.dart' show TakipDurumu, TakipIslemSonucu;
12 |
13 | // Tüm sosyal işlemleri yönlendiren ana servis
14 | class SosyalServis {
15 |   // Gönderi paylaşım hız limiti kontrolü
16 |   static Future<void> gonderiPaylasimHizLimitiKontrol(String uid) =>
17 |       SosyalCore.gonderiPaylasimHizLimitiKontrol(uid);
18 |
19 |   // Gönderi paylaşım zamanını kaydet
20 |   static Future<void> gonderiPaylasimHizLimitiKaydet(String uid) =>
21 |       SosyalCore.gonderiPaylasimHizLimitiKaydet(uid);
22 |
23 |   // Kullanıcı özet bilgisi getir
24 |   static Future<Map<String, dynamic>?> kullaniciOzetGetir(String uid) =>
25 |       SosyalCore.kullaniciOzetGetir(uid);
26 |
27 |   // Etiket metnini normalize et
28 |   static String etiketNormalize(String ham) => EtiketServis.etiketNormalize(ham);
29 |
30 |   // Arama metninden etiket çıkar
31 |   static String? aramaMetnindenEtiket(String arama) =>
32 |       EtiketServis.aramaMetnindenEtiket(arama);
33 |
34 |   // Etiket aramasında gönderi görünürlüğü
35 |   static bool gonderiEtiketAramasindaGorulebilir({
36 |     required Map<String, dynamic> post,
37 |     required String izleyiciUid,
38 |     required Set<String> takipEdilenIdler,
39 |     required Set<String> engelIdleri,
40 |   }) =>
41 |       EtiketServis.gonderiEtiketAramasindaGorulebilir(
42 |         post: post,
43 |         izleyiciUid: izleyiciUid,
44 |         takipEdilenIdler: takipEdilenIdler,
45 |         engelIdleri: engelIdleri,
```

```
46 |       );
47 |
48 |   // Takip edilen kullanıcı id'lerini getir
49 |   static Future<Set<String>> takipEdilenIdleriGetir(String uid) =>
50 |       TakipServis.takipEdilenIdleriGetir(uid);
51 |
52 |   // Hashtag ile gönderileri getir
53 |   static Future<List<QueryDocumentSnapshot<Map<String, dynamic>>>>
54 |       hashtagGonderileriniGetir(
55 |     String hamEtiket, {
56 |     required String izleyiciUid,
57 |     required Set<String> takipEdilenIdler,
58 |     required Set<String> engelIdleri,
59 |     int limit = 30,
60 |   }) =>
```

Begeni, yorum ve mesaj cephesi lib/sosyal_servis.dart:431-524

Post kartlari ve mesaj ekranlari bu metotlari cagirarak alt servislerle konusur.

```
431 |   static Future<bool> begeniDegistir({
432 |     required String postId,
433 |     required String postOwnerId,
434 |     required String currentUserId,
435 |   }) =>
436 |       BegeniYorumServis.begeniDegistir(
437 |         postId: postId,
438 |         postOwnerId: postOwnerId,
439 |         currentUserId: currentUserId,
440 |       );
441 |
442 |   static Future<void> gecersizBegeniKayitlariniTemizle(String uid) =>
443 |       BegeniYorumServis.gecersizBegeniKayitlariniTemizle(uid);
444 |
445 |   // Yorum ekle
446 |   static Future<void> yorumEkle({
447 |     required String postId,
448 |     required String postOwnerId,
449 |     required String currentUserId,
450 |     required String metin,
451 |   }) =>
452 |       BegeniYorumServis.yorumEkle(
453 |         postId: postId,
454 |         postOwnerId: postOwnerId,
455 |         currentUserId: currentUserId,
456 |         metin: metin,
457 |       );
458 |
459 |   // Gönderiyi sil
460 |   static Future<void> gonderiSil(String postId) =>
461 |       BegeniYorumServis.gonderiSil(postId);
462 |
463 |   // İki kullanıcı için sohbet id üret
```

```
464 |   static String sohbetIdUret(String uid1, String uid2) =>
465 |       MesajServis.sohbetIdUret(uid1, uid2);
466 |
467 |   // Sohbet id getir veya oluştur
468 |   static Future<String> sohbetIdGetirVeyaOlustur(String hedefUid) =>
469 |       MesajServis.sohbetIdGetirVeyaOlustur(hedefUid);
470 |
471 |   // Kullanıcının sohbet listesi akışı
472 |   static Stream<QuerySnapshot<Map<String, dynamic>>> sohbetlerim(String uid) =>
473 |       MesajServis.sohbetlerim(uid);
474 |
475 |   // Sohbet mesajları akışı
476 |   static Stream<QuerySnapshot<Map<String, dynamic>>> sohbetMesajlari(
477 |       String chatId) =>
478 |       MesajServis.sohbetMesajlari(chatId);
479 |
480 |   // Mesaj gönder
481 |   static Future<void> mesajGonder({
482 |     required String chatId,
483 |     required String hedefUid,
484 |     required String metin,
485 |     String mesajTipi = "text",
486 |     String postId = "",
487 |   }) =>
488 |       MesajServis.mesajGonder(
489 |         chatId: chatId,
490 |         hedefUid: hedefUid,
491 |         metin: metin,
492 |         mesajTipi: mesajTipi,
493 |         postId: postId,
494 |       );
495 |
496 |   // Gönderiyi mesaj olarak ilet
497 |   static Future<void> gonderiIlet({
498 |     required String postId,
499 |     required String hedefUid,
500 |     required String postMetin,
501 |     required String postKullaniciAdi,
502 |   }) =>
503 |       MesajServis.gonderiIlet(
504 |         postId: postId,
505 |         hedefUid: hedefUid,
506 |         postMetin: postMetin,
507 |         postKullaniciAdi: postKullaniciAdi,
508 |       );
509 |
510 |   // Mesajı sil
511 |   static Future<void> mesajSil({
512 |     required String chatId,
513 |     required String messageId,
514 |     required String mevcutMetin,
515 |   }) =>
516 |       MesajServis.mesajSil(
```

```
517 |         chatId: chatId,
518 |         messageId: messageId,
519 |         mevcutMetin: mevcutMetin,
520 |       );
521 |
522 |   // Sohbeti görüldü olarak işaretle
523 |   static Future<void> sohbetiGorulduIsaretle({
524 |     required String chatId,
```

Kod okurken dikkat et

static: Siniftan nesne olusturmadan metodu cagirabilirsin: SosyalServis.takipEt(...) gibi.

=>: Tek satirlik fonksiyon govdesidir. Soldaki cagriyi sagdaki metoda iletir.

required: Bu named argument verilmeden fonksiyon cagrilamaz demektir.

Map<String, dynamic>: Anahtari String olan, degeri farkli tiplerde olabilen veri sozlugudur; Firestore verisi genelde boyle okunur.

Mini alistirma

1. SosyalServis.begeniDegistir hangi alt servise gidiyor bul.
2. SosyalCore.gonderiPaylasimHizLimitiKontrol kac milisaniye bekletiyor?

8. Robot sistemi

> **Bu bolumun amaci**
> Robot servisi uygulamanin oyunlastirma beynidir. Enerji, mutluluk, uyku, seviye atlama ve sosyal aksiyon maliyetleri burada hesaplanir.

Bu bolumde sunlari ogren

- Model sinifi ile servis sinifi farkini gormek.
- Getter, sabitler, clamp ve parse mantigini anlamak.
- Transaction ile robot verisinin nasil guncellendigini okumaya baslamak.

Robot model ve temel kurallar lib/robot_servis.dart:1-180

RobotDurum veri modelidir; RobotServis ise bu model uzerinde kurallari uygular.

```
1 | // Robot enerji, mutluluk, uyku ve gelişim mantığı.
2 | import 'package:cloud_firestore/cloud_firestore.dart';
3 | import 'package:flutter/foundation.dart';
4 |
5 | // Robot animasyon durumları
6 | enum RobotGorselDurum { normal, mutlu, uzgun, uyku, yorgun }
7 |
8 | // Robot anlık durum modeli
9 | class RobotDurum {
10 |   const RobotDurum({
11 |     required this.isim,
12 |     required this.seviye,
13 |     required this.gelisimPuani,
14 |     required this.enerji,
15 |     required this.mutluluk,
16 |     required this.sonBakim,
17 |     required this.kazanilanSeviyeOdulleri,
18 |     required this.uykuBitisMs,
19 |     required this.sonAktiviteMs,
20 |     required this.sonEnerjiDususMs,
21 |   });
22 |
23 |   final String isim;
24 |   final int seviye;
25 |   final int gelisimPuani;
26 |   final int enerji;
27 |   final int mutluluk;
28 |   final Map<String, dynamic> sonBakim;
29 |   final List<int> kazanilanSeviyeOdulleri;
30 |   final int uykuBitisMs;
31 |   final int sonAktiviteMs;
32 |   final int sonEnerjiDususMs;
33 |
34 |   int get gerekliGelisim => RobotServis.gerekliGelisimPuani(seviye); // Seviye atlama eşiği
```

```
35 |
36 |   // Gelişim çubuğu oranı (0-1)
37 |   double get gelisimOrani {
38 |     if (gerekliGelisim <= 0) return 1;
39 |     return (gelisimPuani / gerekliGelisim).clamp(0.0, 1.0);
40 |   }
41 |
42 |   bool get uyuyor => RobotServis.uyuyorMu(this); // Uyku durumu
43 |
44 |   RobotGorselDurum get gorselDurum => RobotServis.gorselDurum(this); // Animasyon durumu
45 | }
46 |
47 | // Robot geliştirme işlemi sonucu
48 | class RobotGelistirSonuc {
49 |   const RobotGelistirSonuc({
50 |     required this.seviyeAtladi,
51 |     required this.eskiSeviye,
52 |     required this.yeniSeviye,
53 |     required this.yeniSeviyeOdulleri,
54 |   });
55 |
56 |   final bool seviyeAtladi;
57 |   final int eskiSeviye;
58 |   final int yeniSeviye;
59 |   final List<int> yeniSeviyeOdulleri;
60 | }
61 |
62 | // Zaman senkronizasyonu sonucu
63 | class RobotSenkronSonuc {
64 |   const RobotSenkronSonuc({
65 |     required this.robot,
66 |     required this.mesajlar,
67 |   });
68 |
69 |   final RobotDurum robot;
70 |   final List<String> mesajlar;
71 | }
72 |
73 | // Robot oyun mekaniği servisi
74 | class RobotServis {
75 |   RobotServis._();
76 |
77 |   static const gelistirmeMaliyeti = 20; // Geliştirme puan maliyeti
78 |   static const odaAcilisSeviyesi = 5; // Robot odası açılış seviyesi
79 |   static const uyuMaxEnerji = 50; // Uyku için max enerji eşiği
80 |   static const uyuErkenUyandirmaMinEnerji = 30; // Erken uyandırma min enerji
81 |   static const uyuSureMs = 4 * 60 * 60 * 1000; // Uyku süresi (4 saat)
82 |   static const uykuEnerjiDakikaBasina = 0.2; // Uykuda dakika başı enerji
83 |   static const uykuMaxEnerjiArtisi = 50; // Uykuda max enerji artışı
84 |   static const etkilesimEnerjiGonderi = 5; // Gönderi enerji maliyeti
85 |   static const etkilesimEnerjiYorum = 3; // Yorum enerji maliyeti
86 |   static const etkilesimEnerjiBegeni = 1; // Beğeni enerji maliyeti
87 |   /// Uygulama kapalıyken mutluluğun sıfıra inme süresi (dakika).
```

```
  static const mutlulukUygulamaDisiDakika = 120;
  /// Uygulama açıkken etkileşim yoksa mutluluğun sıfıra inme süresi (dakika).
  static const mutlulukEtkilesimsizDakika = 240;
  static const uyanincaMutlulukBonus = 5; // Uyanınca mutluluk bonusu
  static const _dakikaMs = 60 * 1000;

  // Eski bakım anahtarlarını temizler
  static void _sonBakimTemizle(Map<String, dynamic> robot) {
    final sonBakimRaw = robot['sonBakim'];
    if (sonBakimRaw is! Map) return;

    final sonBakim = Map<String, dynamic>.from(sonBakimRaw);
    var degisti = false;

    for (final key in ['yemek', 'uyu', 'besle', 'oyna']) {
      if (sonBakim.remove(key) != null) {
        degisti = true;
      }
    }

    if (degisti) {
      robot['sonBakim'] = sonBakim;
    }
  }

  // Robot odası açık mı (seviye 5+)
  static bool odaAcik(int seviye) => seviye >= odaAcilisSeviyesi;

  static final FirebaseFirestore _db = FirebaseFirestore.instance;

  static const _odulSeviyeleri = [2, 3, 4, 5, 7, 10];

  static int gerekliGelisimPuani(int seviye) => seviye < 5 ? 100 : 500;

  static int _int(dynamic value, int varsayilan) {
    if (value is int) return value;
    if (value is num) return value.toInt();
    if (value is String) {
      final parsed = int.tryParse(value.trim());
      if (parsed != null) return parsed;
    }
    return varsayilan;
  }

  static int _clampStat(int value) => value.clamp(0, 100);

  static int _simdiMs() => DateTime.now().millisecondsSinceEpoch;

  // Firestore verisini RobotDurum'a çevirir
  static RobotDurum parse(dynamic raw) {
    final map = raw is Map
        ? Map<String, dynamic>.from(raw)
        : <String, dynamic>{};
```

```
141 |     final sonBakimRaw = map['sonBakim'];
142 |     final sonBakim = sonBakimRaw is Map
143 |         ? Map<String, dynamic>.from(sonBakimRaw)
144 |         : <String, dynamic>{};
145 |     final odulRaw = map['kazanilanSeviyeOdulleri'];
146 |     final oduller = odulRaw is List
147 |         ? odulRaw.map((e) => _int(e, 0)).where((e) => e > 0).toList()
148 |         : <int>[];
149 |     final simdi = _simdiMs();
150 |
151 |     return RobotDurum(
152 |       isim: (map['isim'] ?? 'Mini Vibe').toString(),
153 |       seviye: _int(map['seviye'], 1),
154 |       gelisimPuani: _int(map['gelisimPuani'], 0),
155 |       enerji: _clampStat(_int(map['enerji'], 50)),
156 |       mutluluk: _clampStat(_int(map['mutluluk'], 50)),
157 |       sonBakim: sonBakim,
158 |       kazanilanSeviyeOdulleri: oduller,
159 |       uykuBitisMs: _int(map['uykuBitisMs'], 0),
160 |       sonAktiviteMs: _int(map['sonAktiviteMs'], simdi),
161 |       sonEnerjiDususMs: _int(map['sonEnerjiDususMs'], simdi),
162 |     );
163 |   }
164 |
165 |   // Robot şu an uyuyor mu
166 |   static bool uyuyorMu(RobotDurum robot) =>
167 |       robot.uykuBitisMs > _simdiMs();
168 |
169 |   // Erken uyandırma mümkün mü
170 |   static bool erkenUyandirabilir(RobotDurum robot) =>
171 |       uyuyorMu(robot) && robot.enerji > uyuErkenUyandirmaMinEnerji;
172 |
173 |   static Duration? uykuKalanSure(RobotDurum robot) {
174 |     if (!uyuyorMu(robot)) return null;
175 |     return Duration(milliseconds: robot.uykuBitisMs - _simdiMs());
176 |   }
177 |
178 |   // Duruma göre animasyon durumu seçer
179 |   static RobotGorselDurum gorselDurum(RobotDurum robot) {
180 |     if (!odaAcik(robot.seviye)) return RobotGorselDurum.normal;
```

Etkilesim enerji maliyeti lib/robot_servis.dart:189-248

Gonderi, yorum ve begeni gibi aksiyonlar robot enerjisini tuketir; enerji yoksa aksiyon engellenir.

```
189 |   static int etkilesimEnerjiMaliyeti(String aksiyonTipi) {
190 |     switch (aksiyonTipi) {
191 |       case 'gonderi':
192 |         return etkilesimEnerjiGonderi;
193 |       case 'yorum':
194 |         return etkilesimEnerjiYorum;
195 |       case 'begeni':
196 |         return etkilesimEnerjiBegeni;
```

```
197 |       default:
198 |         return 1;
199 |     }
200 |   }
201 |
202 |   // Enerji/uyku engeli varsa hata fırlatır
203 |   static void _etkilesimEngeli(RobotDurum robot, {required int enerjiMaliyeti}) {
204 |     if (!odaAcik(robot.seviye)) return;
205 |     if (uyuyorMu(robot)) {
206 |       throw '${robot.isim} uyuyor. Robot uyanana kadar bu işlem kapalı.';
207 |     }
208 |     if (enerjiMaliyeti <= 0) {
209 |       if (robot.enerji <= 0) {
210 |         throw '${robot.isim} çok yoruldu. Enerji bitene kadar etkileşim yapılamaz. '
211 |             'Uyutarak enerji toplayabilirsin.';
212 |       }
213 |       return;
214 |     }
215 |     if (robot.enerji < enerjiMaliyeti) {
216 |       throw 'Bu işlem için en az $enerjiMaliyeti enerji gerekir. '
217 |           '(Mevcut: ${robot.enerji})';
218 |     }
219 |   }
220 |
221 |   // Firestore'dan robot durumunu okuyup engel kontrolü yapar
222 |   static Future<void> etkilesimEngelliyseAt(
223 |     String uid, {
224 |     required String aksiyonTipi,
225 |   }) async {
226 |     if (uid.isEmpty) return;
227 |     final snap = await _db.collection('users').doc(uid).get();
228 |     etkilesimEngeliVeriden(
229 |       snap.data(),
230 |       aksiyonTipi: aksiyonTipi,
231 |     );
232 |   }
233 |
234 |   static void etkilesimEngeliVeriden(
235 |     Map<String, dynamic>? userData, {
236 |     required String aksiyonTipi,
237 |   }) {
238 |     if (userData == null) return;
239 |     final robot = parse(userData['robot']);
240 |     _etkilesimEngeli(
241 |       robot,
242 |       enerjiMaliyeti: etkilesimEnerjiMaliyeti(aksiyonTipi),
243 |     );
244 |   }
245 |
246 |   // Robot ruh hali mesajını döndürür
247 |   static String ruhHaliMesaji(RobotDurum robot) {
248 |     if (!odaAcik(robot.seviye)) {
```

Senkron, gelistir, uyut ve sosyal aksiyon lib/robot_servis.dart:512-690

Firebase transaction bolumleri burada yogunlasir; bu parcayi ikinci okumada daha detayli incele.

```
512 |   static Future<RobotSenkronSonuc?> senkronizeEt(String uid) async {
513 |     if (uid.isEmpty) return null;
514 |     final ref = _db.collection('users').doc(uid);
515 |     try {
516 |       return await _db.runTransaction((tx) async {
517 |         final snap = await tx.get(ref);
518 |         if (!snap.exists) return null;
519 |         final robot = _robotHaritasi(snap.data()?['robot']);
520 |         if (!odaAcik(_int(robot['seviye'], 1))) {
521 |           return RobotSenkronSonuc(robot: parse(robot), mesajlar: const []);
522 |         }
523 |         final mesajlar =
524 |             _zamanGuncelle(robot, _simdiMs(), uygulamaAcildi: true);
525 |         tx.update(ref, {'robot': robot});
526 |         return RobotSenkronSonuc(
527 |           robot: parse(robot),
528 |           mesajlar: mesajlar,
529 |         );
530 |       });
531 |     } catch (e) {
532 |       debugPrint('Robot senkronizasyon hatası: $e');
533 |       return null;
534 |     }
535 |   }
536 |
537 |   /// Uygulama açıkken dakikalık mutluluk düşüşü (etkileşimsiz 4 saat kuralı).
538 |   static Future<void> dakikaSenkronizeEt(String uid) async {
539 |     if (uid.isEmpty) return;
540 |     final ref = _db.collection('users').doc(uid);
541 |     try {
542 |       await _db.runTransaction((tx) async {
543 |         final snap = await tx.get(ref);
544 |         if (!snap.exists) return;
545 |         final robot = _robotHaritasi(snap.data()?['robot']);
546 |         if (!odaAcik(_int(robot['seviye'], 1))) return;
547 |         _zamanGuncelle(robot, _simdiMs());
548 |         tx.update(ref, {'robot': robot});
549 |       });
550 |     } catch (e) {
551 |       debugPrint('Robot dakika senkron hatası: $e');
552 |     }
553 |   }
554 |
555 |   // Puan harcayarak robotu geliştirir
556 |   static Future<RobotGelistirSonuc> gelistir(String uid) async {
557 |     final ref = _db.collection('users').doc(uid);
558 |     return _db.runTransaction((tx) async {
559 |       final snap = await tx.get(ref);
560 |       final data = snap.data() ?? {};
561 |       final puan = _int(data['puan'], 0);
```

```
562 |       if (puan < gelistirmeMaliyeti) {
563 |         throw 'Geliştirmek için en az $gelistirmeMaliyeti puan gerekir.';
564 |       }
565 |
566 |       final robot = _robotHaritasi(data['robot']);
567 |       final simdi = _simdiMs();
568 |       _zamanGuncelle(robot, simdi, etkilesimKaydedildi: true);
569 |       _uykuEngeli(robot);
570 |       _etkilesimEngeli(parse(robot), enerjiMaliyeti: 0);
571 |
572 |       var seviye = _int(robot['seviye'], 1);
573 |       final eskiSeviye = seviye;
574 |       var gelisimPuani = _int(robot['gelisimPuani'], 0);
575 |       gelisimPuani += gelistirmeMaliyeti;
576 |
577 |       var gerekli = gerekliGelisimPuani(seviye);
578 |       var seviyeAtladi = false;
579 |       while (gelisimPuani >= gerekli) {
580 |         gelisimPuani -= gerekli;
581 |         seviye += 1;
582 |         seviyeAtladi = true;
583 |         gerekli = gerekliGelisimPuani(seviye);
584 |       }
585 |
586 |       robot['seviye'] = seviye;
587 |       robot['gelisimPuani'] = gelisimPuani;
588 |       if (odaAcik(seviye)) {
589 |         robot['mutluluk'] = _clampStat(_int(robot['mutluluk'], 50) + 8);
590 |       }
591 |
592 |       final yeniOduller = seviyeAtladi
593 |           ? _seviyeOdulleriniIsle(robot, seviye)
594 |           : <int>[];
595 |
596 |       tx.update(ref, {
597 |         'puan': FieldValue.increment(-gelistirmeMaliyeti),
598 |         'robot': robot,
599 |       });
600 |
601 |       return RobotGelistirSonuc(
602 |         seviyeAtladi: seviyeAtladi,
603 |         eskiSeviye: eskiSeviye,
604 |         yeniSeviye: seviye,
605 |         yeniSeviyeOdulleri: yeniOduller,
606 |       );
607 |     });
608 |   }
609 |
610 |   // Robot ismini günceller
611 |   static Future<String> isimGuncelle(String uid, String yeniIsim) async {
612 |     final temiz = yeniIsim.trim();
613 |     if (temiz.isEmpty || temiz.length > 24) {
614 |       throw 'Robot ismi 1-24 karakter olmalı.';
```

```
615 |     }
616 |     await _db.collection('users').doc(uid).update({'robot.isim': temiz});
617 |     return temiz;
618 |   }
619 |
620 |   // Uyuyan robotu erken uyandırır
621 |   static Future<String> uyandir(String uid) async {
622 |     final ref = _db.collection('users').doc(uid);
623 |     return _db.runTransaction((tx) async {
624 |       final snap = await tx.get(ref);
625 |       final robot = _robotHaritasi(snap.data()?['robot']);
626 |       if (!odaAcik(_int(robot['seviye'], 1))) {
627 |         throw 'Robot odası $odaAcilisSeviyesi. seviyede açılır.';
628 |       }
629 |       final simdi = _simdiMs();
630 |       _zamanGuncelle(robot, simdi, etkilesimKaydedildi: true);
631 |
632 |       if (_int(robot['uykuBitisMs'], 0) <= simdi) {
633 |         throw 'Robot zaten uyanık.';
634 |       }
635 |
636 |       final enerji = _int(robot['enerji'], 50);
637 |       if (enerji <= uyuErkenUyandirmaMinEnerji) {
638 |         throw 'Enerji %$uyuErkenUyandirmaMinEnerji altındayken erken uyandırılamaz.';
639 |       }
640 |
641 |       _robotUyandir(robot);
642 |       tx.update(ref, {'robot': robot});
643 |       return '${robot['isim'] ?? 'Robot'} uyandı!';
644 |     });
645 |   }
646 |
647 |   // Robotu uyutmaya başlatır
648 |   static Future<String> uyuYap(String uid) async {
649 |     final ref = _db.collection('users').doc(uid);
650 |     return _db.runTransaction((tx) async {
651 |       final snap = await tx.get(ref);
652 |       final robot = _robotHaritasi(snap.data()?['robot']);
653 |       if (!odaAcik(_int(robot['seviye'], 1))) {
654 |         throw 'Robot odası $odaAcilisSeviyesi. seviyede açılır.';
655 |       }
656 |       final simdi = _simdiMs();
657 |       _zamanGuncelle(robot, simdi, etkilesimKaydedildi: true);
658 |
659 |       final enerji = _int(robot['enerji'], 50);
660 |       if (enerji > uyuMaxEnerji) {
661 |         throw 'Enerji %51 ve üzerindeyken robot uyuyamaz.';
662 |       }
663 |       if (_int(robot['uykuBitisMs'], 0) > simdi) {
664 |         throw 'Robot zaten uyuyor.';
665 |       }
666 |
667 |       robot['uykuBaslangicEnerji'] = enerji;
```

```
668 |       robot['uykuBaslangicMs'] = simdi;
669 |       robot['uykuBitisMs'] = simdi + uyuSureMs;
670 |
671 |       tx.update(ref, {'robot': robot});
672 |       return 'Robot uyumaya başladı. 4 saatte enerji %$uykuMaxEnerjiArtisi dolacak.';
673 |     });
674 |   }
675 |
676 |   // Sosyal aksiyon sonrası enerji/mutluluk günceller
677 |   static Future<void> sosyalAksiyon(String uid, String tip) async {
678 |     if (uid.isEmpty) return;
679 |     final ref = _db.collection('users').doc(uid);
680 |     try {
681 |       await _db.runTransaction((tx) async {
682 |         final snap = await tx.get(ref);
683 |         if (!snap.exists) return;
684 |         final robot = _robotHaritasi(snap.data()?['robot']);
685 |         if (!odaAcik(_int(robot['seviye'], 1))) return;
686 |         final simdi = _simdiMs();
687 |         _zamanGuncelle(robot, simdi, etkilesimKaydedildi: true);
688 |         if (_int(robot['uykuBitisMs'], 0) > simdi) return;
689 |
690 |         final maliyet = etkilesimEnerjiMaliyeti(tip);
```

Robot durum cubugu widget'i lib/widgets/robot_durum_bari.dart:1-64

Robot servisinden gelen enerji ve mutluluk degerleri ekranda bu widget ile cubuga donusur.

```
1 | // Robot enerji/mutluluk ilerleme çubuğu.
2 | import 'package:flutter/material.dart';
3 | import 'package:vibe_life/app_theme.dart';
4 |
5 | // Etiketli yüzdelik durum çubuğu
6 | class RobotDurumBari extends StatelessWidget {
7 |   const RobotDurumBari({
8 |     super.key,
9 |     required this.etiket,
10 |     required this.deger,
11 |     required this.renk,
12 |     this.ters = false,
13 |     this.compact = false,
14 |   });
15 |
16 |   final String etiket; // Gösterilecek isim (Enerji, Mutluluk)
17 |   final int deger; // 0-100 arası değer
18 |   final Color renk; // Çubuk rengi
19 |   final bool ters; // Doluluk tersine çevrilsin mi
20 |   final bool compact; // Küçük boyut modu
21 |
22 |   @override
23 |   Widget build(BuildContext context) {
24 |     final theme = Theme.of(context);
25 |     final vibe = context.vibe;
```

```
    final gosterim = ters ? (100 - deger.clamp(0, 100)) : deger.clamp(0, 100);
    final oran = gosterim / 100.0;

    return Padding(
      padding: EdgeInsets.only(bottom: compact ? 6 : 10),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Text(
                etiket,
                style: TextStyle(
                  color: vibe.mutedText,
                  fontSize: compact ? 11 : 12,
                  fontWeight: FontWeight.w600,
                ),
              ),
              const Spacer(),
              Text(
                '$deger',
                style: TextStyle(
                  color: theme.colorScheme.onSurface,
                  fontSize: compact ? 11 : 12,
                ),
              ),
            ],
          ),
          SizedBox(height: compact ? 4 : 6),
          ClipRRect(
            borderRadius: BorderRadius.circular(8),
            child: LinearProgressIndicator(
              value: oran,
              minHeight: compact ? 6 : 8,
              backgroundColor: vibe.inputFill,
              valueColor: AlwaysStoppedAnimation<Color>(renk),
            ),
          ),
        ],
```

Kod okurken dikkat et

model: Veriyi tutan siniftir. RobotDurum sadece robotun anlik alanlarini tasir.

servis: Is kurallarini calistiran siniftir. RobotServis seviye, enerji ve uyku hesabini yapar.

clamp: Degeri belirli aralikta tutar. Enerji ve mutluluk 0-100 disina cikmasin diye kullanilir.

transaction: Firestore'da once oku, sonra okudugun degerlere gore guvenli yaz demektir.

Mini alistirma

1. Gonderi, yorum ve begeni enerji maliyetlerini bul.
2. Robot odasi hangi seviyede aciliyor?

3. RobotDurum icindeki final alanlari neden sonradan degistirilmiyor?

9. Firestore kurallarini okuma

> **Bu bolumun amaci**
> Uygulamanin gercek guvenligi sadece UI tarafinda degil Firestore rules dosyasindadir. Bu dosya kim neyi okuyabilir, yazabilir ve silebilir sorusunu cevaplar.

Bu bolumde sunlari ogren

- rules dosyasinda function ve match kavramlarini anlamak.
- isSignedIn, isOwner, canReadPostData gibi yardimci kurallari okumak.
- users ve posts koleksiyonlarinin izinlerini takip etmek.

Kimlik ve gorunurluk yardimcilari firestore.rules:1-120

Bu fonksiyonlar alttaki allow kurallarinda tekrar tekrar kullanilir.

```
rules_version = '2';

// Vibe Life — Firestore güvenlik kuralları
// Koleksiyonlar: usernames, users (+ alt koleksiyonlar), posts (+ likes/comments),
// reports, chats (+ messages)

service cloud.firestore {
  match /databases/{database}/documents {

    // ---------------------------------------------------------------------
    // Kimlik ve roller
    // ---------------------------------------------------------------------

    function isSignedIn() {
      return request.auth != null;
    }

    function isOwner(uid) {
      return isSignedIn() && request.auth.uid == uid;
    }

    function isModerator() {
      return isSignedIn()
        && get(/databases/$(database)/documents/users/$(request.auth.uid)).data.isModerator == true;
    }

    // ---------------------------------------------------------------------
    // Alan yardımcıları
    // ---------------------------------------------------------------------

    function intField(data, field, fallback) {
      return data[field] is number ? data[field] : fallback;
    }

```

```
35 |     function changedOnly(fields) {
36 |       return request.resource.data.diff(resource.data).affectedKeys().hasOnly(fields);
37 |     }
38 |
39 |     function intChangedBy(field, delta) {
40 |       return intField(request.resource.data, field, 0)
41 |         == intField(resource.data, field, 0) + delta;
42 |     }
43 |
44 |     function userCountersNonNegative() {
45 |       return intField(request.resource.data, 'puan', 0) >= 0
46 |         && intField(request.resource.data, 'gonderiSayisi', 0) >= 0
47 |         && intField(request.resource.data, 'begeniSayisi', 0) >= 0
48 |         && intField(request.resource.data, 'yorumSayisi', 0) >= 0;
49 |     }
50 |
51 |     function robotSeviyeFrom(data) {
52 |       return data.robot is map && data.robot.seviye is number
53 |         ? data.robot.seviye
54 |         : 1;
55 |     }
56 |
57 |     // ------------------------------------------------------------------------
58 |     // Sosyal grafik
59 |     // ------------------------------------------------------------------------
60 |
61 |     function isFollower(ownerUid, followerUid) {
62 |       return exists(/databases/$(database)/documents/users/$(ownerUid)/followers/$(followerUid));
63 |     }
64 |
65 |     function isFollowing(followerUid, targetUid) {
66 |       return exists(/databases/$(database)/documents/users/$(followerUid)/following/$(targetUid));
67 |     }
68 |
69 |     function userIsPrivate(uid) {
70 |       return exists(/databases/$(database)/documents/users/$(uid))
71 |         && get(/databases/$(database)/documents/users/$(uid)).data.hesapGizli == true;
72 |     }
73 |
74 |     function canReadSocialGraph(uid) {
75 |       return isSignedIn()
76 |         && (
77 |           isOwner(uid)
78 |           || isModerator()
79 |           || userIsPrivate(uid) != true
80 |           || isFollower(uid, request.auth.uid)
81 |         );
82 |     }
83 |
84 |     // ------------------------------------------------------------------------
85 |     // Gönderi görünürlüğü (feed sorgularıyla uyumlu)
86 |     // ------------------------------------------------------------------------
87 |
```

```
88 |     function canReadPostData(data) {
89 |       // ownerHesapGizli sorgularla uyumlu; canlı userIsPrivate() sorgu sonuçlarını bozar.
90 |       return isSignedIn()
91 |         && (
92 |           data.raporNedeniyleGizli != true
93 |           || data.userId == request.auth.uid
94 |           || isModerator()
95 |         )
96 |         && (
97 |           data.ownerHesapGizli != true
98 |           || data.userId == request.auth.uid
99 |           || isFollower(data.userId, request.auth.uid)
100 |          || isModerator()
101 |        );
102 |    }
103 |
104 |    function begendigiPostMu(postId) {
105 |      return isSignedIn()
106 |        && (
107 |          exists(/databases/$(database)/documents/users/$(request.auth.uid)/likedPosts/$(postId))
108 |          || exists(/databases/$(database)/documents/posts/$(postId)/likes/$(request.auth.uid))
109 |        );
110 |    }
111 |
112 |    function canReadPost(postId) {
113 |      return canReadPostData(
114 |        get(/databases/$(database)/documents/posts/$(postId)).data
115 |      ) || begendigiPostMu(postId);
116 |    }
117 |
118 |    function postOwner(postId) {
119 |      return get(/databases/$(database)/documents/posts/$(postId)).data.userId;
120 |    }
```

users ve posts izinleri firestore.rules:638-760

Kullanici ve gonderi koleksiyonlarinda okuma/yazma izinleri burada tanimlanir.

```
638 |    match /users/{userId} {
639 |      // Keşfet / liderlik listeleri tüm profilleri okur; içerik gizliliği post kurallarında.
640 |      allow get, list: if isSignedIn();
641 |      allow create: if isOwner(userId) && userCreateOk();
642 |      allow update: if (isOwner(userId) && ownerUserUpdateOk())
643 |        || ownerScoreUpdateOk(userId)
644 |        || (isOwner(userId) && haftalikGorevUpdateOk())
645 |        || (isModerator()
646 |          && request.resource.data.diff(resource.data).affectedKeys()
647 |              .hasOnly(['raporNedeniyleGizli', 'raporGuncellenmeTarihi']));
648 |      allow delete: if isOwner(userId) || isModerator();
649 |
650 |      match /following/{targetId} {
651 |        allow read: if canReadSocialGraph(userId)
652 |          || request.auth.uid == userId
```

```
653 |           || (request.auth.uid == targetId
654 |             && bekleyenTakipIstegiVar(targetId, userId));
655 |         allow create: if followingCreateOk(userId, targetId);
656 |         allow update: if followingCreateOk(userId, targetId);
657 |         allow delete: if isOwner(userId) || request.auth.uid == targetId || isModerator();
658 |       }
659 |
660 |       match /followers/{followerId} {
661 |         allow read: if canReadSocialGraph(userId)
662 |           || request.auth.uid == userId
663 |           || (request.auth.uid == followerId
664 |             && bekleyenTakipIstegiVar(userId, followerId));
665 |         allow create: if followerCreateOk(userId, followerId);
666 |         allow update: if followerCreateOk(userId, followerId);
667 |         allow delete: if isOwner(userId) || request.auth.uid == followerId || isModerator();
668 |       }
669 |
670 |       match /followRequests/{requesterId} {
671 |         allow read: if isOwner(userId) || request.auth.uid == requesterId;
672 |         allow create: if request.auth.uid == requesterId
673 |           && request.resource.data.userId == requesterId
674 |           && get(/databases/$(database)/documents/users/$(userId)).data.hesapGizli == true;
675 |         allow delete: if isOwner(userId) || request.auth.uid == requesterId;
676 |         allow update: if false;
677 |       }
678 |
679 |       match /notifications/{notificationId} {
680 |         allow read: if isOwner(userId);
681 |         allow create: if isSignedIn() && notificationCreateOk();
682 |         allow update: if isOwner(userId)
683 |           && request.resource.data.diff(resource.data).affectedKeys().hasOnly(['okundu']);
684 |         allow delete: if isOwner(userId)
685 |           || (isSignedIn() && resource.data.gonderenUid == request.auth.uid);
686 |       }
687 |
688 |       match /blocked/{blockedId} {
689 |         allow read: if isOwner(userId);
690 |         allow create, update: if isOwner(userId);
691 |         allow delete: if isOwner(userId) || request.auth.uid == blockedId;
692 |       }
693 |
694 |       match /blockedBy/{blockerId} {
695 |         allow read: if isOwner(userId);
696 |         allow create, update: if request.auth.uid == blockerId;
697 |         allow delete: if request.auth.uid == blockerId || isOwner(userId);
698 |       }
699 |
700 |       match /likedPosts/{postId} {
701 |         allow read: if isOwner(userId);
702 |         allow create: if isOwner(userId)
703 |           && request.resource.data.postId == postId
704 |           && existsAfter(/databases/$(database)/documents/posts/$(postId)/likes/$(userId))
705 |           && (!('puanAlindi' in request.resource.data)
```

```
706 |               || request.resource.data.puanAlindi is bool);
707 |         allow delete: if isOwner(userId);
708 |         allow update: if false;
709 |       }
710 |
711 |       match /followRewarded/{targetId} {
712 |         allow read: if isOwner(userId) || request.auth.uid == targetId;
713 |         allow create: if followRewardedCreateOk(userId, targetId);
714 |         allow delete: if isOwner(userId);
715 |         allow update: if isOwner(userId)
716 |           && request.resource.data.diff(resource.data).affectedKeys().hasOnly(['puanVerildi'])
717 |           && request.resource.data.puanVerildi == true
718 |           && request.resource.data.targetUid == resource.data.targetUid
719 |           && (
720 |             resource.data.puanVerildi == false
721 |             || !('puanVerildi' in resource.data)
722 |           );
723 |       }
724 |     }
725 |
726 |     // ================================================================================
727 |     // posts — akış, beğeni, yorum
728 |     // Sorgular:
729 |     //   Keşfet: ownerHesapGizli==false, raporNedeniyleGizli==false
730 |     //   Takip: userId==X, raporNedeniyleGizli==false
731 |     //   Profil: userId==X
732 |     // ================================================================================
733 |
734 |     match /posts/{postId} {
735 |       allow read: if canReadPostData(resource.data) || begendigiPostMu(postId);
736 |       allow create: if isSignedIn() && postCreateOk();
737 |       allow update: if isSignedIn() && (
738 |         (resource.data.userId == request.auth.uid
739 |           && (ownerPostUpdateOk() || postCounterUpdateOk(postId)))
740 |         || (resource.data.userId != request.auth.uid && (
741 |           postCounterUpdateOk(postId) || postReportUpdateOk(postId)
742 |         ))
743 |         || (isModerator() && moderatorContentUpdateOk())
744 |       );
745 |       allow delete: if isSignedIn() && resource.data.userId == request.auth.uid;
746 |
747 |       match /likes/{likeId} {
748 |         allow read: if isSignedIn()
749 |           && (canReadPost(postId) || likeId == request.auth.uid);
750 |         allow create: if isSignedIn()
751 |           && canReadPost(postId)
752 |           && request.auth.uid == likeId
753 |           && request.resource.data.userId == request.auth.uid;
754 |         allow delete: if isSignedIn()
755 |           && (request.auth.uid == likeId || postOwner(postId) == request.auth.uid);
756 |         allow update: if false;
757 |       }
758 |
```

```
759 |       match /comments/{commentId} {
760 |         allow read: if canReadPost(postId);
```

Kod okurken dikkat et

request.auth: Istek yapan kullanicinin oturum bilgisidir.

resource.data: Veritabaninda zaten duran mevcut belgenin verisidir.

request.resource.data: Yazma isleminden sonra belgenin alacagi yeni veridir.

allow read/create/update/delete: Okuma, olusturma, guncelleme ve silme izinlerini ayri ayri belirler.

exists / get: Firestore icinde baska belge var mi veya verisi ne diye kontrol eder.

Mini alistirma

1. posts icin allow create hangi fonksiyona baglanmis bul.
2. Gizli hesap postlarini kimin okuyabilecegini canReadPostData uzerinden acikla.

Büyük dosyalar icin okuma haritasi

> **Neden bu bolum var?**
> Projede bazi dosyalar cok uzun. Bu dosyalari satir satir okumadan once hangi parcaya ne zaman bakacagini bilmek gerekir.

Kavram	Bu projede nasil okunur?
`profile_page.dart`	Kendi profil ekranin. Ayarlar, hesap silme, robot ismi, profil duzenleme ve kullanici gonderileri burada. En sona birak.
`messages_page.dart`	Sohbet listesi ve birebir mesajlasma. StreamBuilder, local okundu bilgisi ve Firestore chat yapisini ogrendikten sonra oku.
`user_profile_page.dart`	Baska kullanicinin profili. Takip, engel, mesaj ve gizli hesap davranisini anlamak icin takip_servis sonrasinda oku.
`feed/feed_content.dart`	Ana akis sorgulari ve listeleme. Firestore query, pagination ve PostCard yapisini ogrendikten sonra oku.
`notifications_page.dart`	Bildirim listesi ve takip istegi aksiyonlari. BildirimServis ve TakipServis mantigindan sonra oku.

AI'ye sorarken kullanacagin hazir kalip

```
Ben yazilimi VibeLife projesi uzerinden ogreniyorum.
Asagidaki kodu Dart/Flutter bilmeyen birine anlatir gibi acikla.
Her satirdaki kelimeleri, noktalari, parantezleri, ? ve ! isaretlerini tek tek anlat.
Once kodun genel amacini, sonra satir satir ne yaptigini, sonra da 5 alistirma sorusu ver.
```

Sonraki cilt onerisi

Bu PDF Cilt 1 olarak temel okuma sirani ve ana akislari kurar. Bir sonraki PDF icin en dogru konu Cilt 2: Giris-Kayit ve Profil olacaktir. O ciltte login_page.dart, register_page.dart, kayit_servis.dart ve profile_page.dart daha uzun ve satir satir islenebilir.

Ucuncu cilt ise Sosyal Sistem olabilir: takip, begeni, yorum, bildirim, mesajlasma ve gizli hesap kurallari birlikte okunur.